UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 6

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME :SULTAN RUHUL KUDDUS NAYAN

STUDENT ID : 23101159

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 31- 08 - 2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

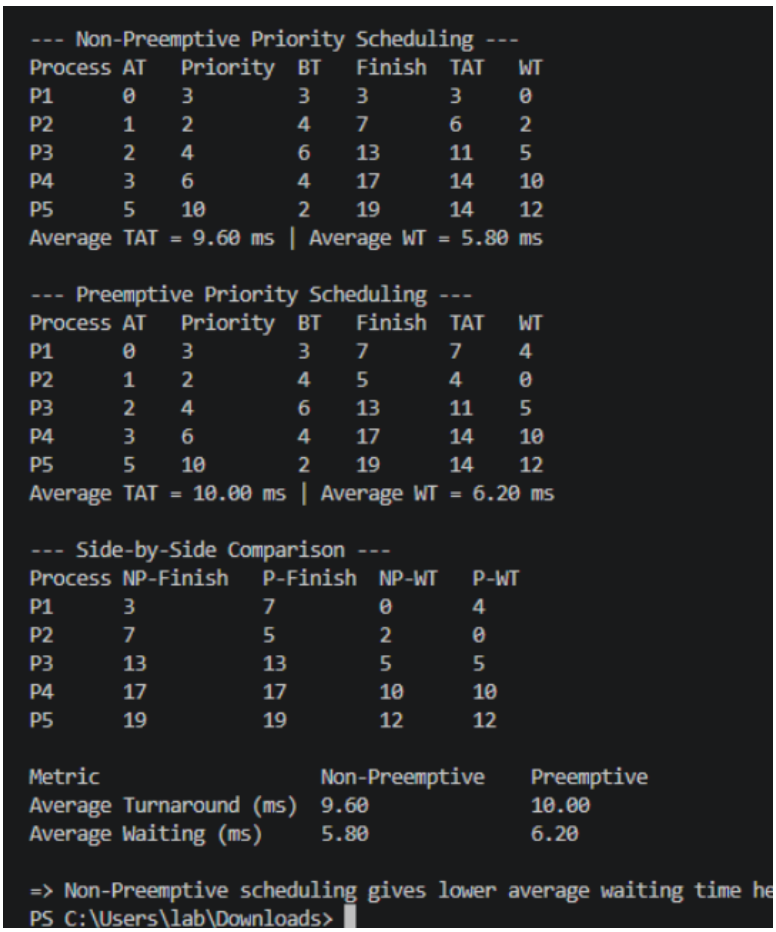
1. Source Code:

```
processes_base = [
    {"pid": "P1", "at": 0, "priority": 3, "bt": 3},
    {"pid": "P2", "at": 1, "priority": 2, "bt": 4},
    {"pid": "P3", "at": 2, "priority": 4, "bt": 6},
    {"pid": "P4", "at": 3, "priority": 6, "bt": 4},
    {"pid": "P5", "at": 5, "priority": 10, "bt": 2},
]


def non_preemptive(processes):
    n = len(processes)
    completed, completed_pids = [], set()
    current_time = 0

    while len(completed) < n:
        ready = [p for p in processes
                 if p["at"] <= current_time and p["pid"] not in
completed_pids]
        if not ready:
            current_time = min(p["at"] for p in processes if p["pid"] not
in completed_pids)
            continue

        current = min(ready, key=lambda p: (p["priority"], p["at"]))
        start = current_time
        finish = start + current["bt"]
        tat = finish - current["at"]
        wt = tat - current["bt"]

        completed.append({**current, "finish": finish, "turnaround": tat,
"waiting": wt})
        completed_pids.add(current["pid"])
        current_time = finish

    return completed


def preemptive(processes):
    n = len(processes)
    remaining = {p["pid"]: p["bt"] for p in processes}
    finish_time = {}
    current_time = 0
    completed = 0

    while completed < n:
        ready = [p for p in processes
                 if p["at"] <= current_time and remaining[p["pid"]] > 0]
        if not ready:
            current_time += 1
            continue

        current = min(ready, key=lambda p: (p["priority"], p["at"]))
```

```
        pid = current["pid"]
        remaining[pid] -= 1
        current_time += 1

        if remaining[pid] == 0:
            finish_time[pid] = current_time
            completed += 1

    results = []
    for p in processes:
        ct = finish_time[p["pid"]]
        tat = ct - p["at"]
        wt = tat - p["bt"]
        results.append({**p, "finish": ct, "turnaround": tat, "waiting":
wt})
    return results


def print_table(title, results):
    n = len(results)
    print(f"\n--- {title} ---")

print(f"{'Process':<8}{'AT':<5}{'Priority':<10}{'BT':<5}{'Finish':<8}{'TAT
':<6}{'WT':<5}")
    total_tat = total_wt = 0
    for r in results:
        print(f"{r['pid']:<8}{r['at']:<5}{r['priority']:<10}{r['bt']:<5}"
              f"{r['finish']:<8}{r['turnaround']:<6}{r['waiting']:<5}")
        total_tat += r["turnaround"]
        total_wt += r["waiting"]
    avg_tat = total_tat / n
    avg_wt = total_wt / n
    print(f"Average TAT = {avg_tat:.2f} ms | Average WT = {avg_wt:.2f}
ms")
    return avg_tat, avg_wt


np_results = non_preemptive([p.copy() for p in processes_base])
pre_results = preemptive([p.copy() for p in processes_base])

np_avg_tat, np_avg_wt = print_table("Non-Preemptive Priority Scheduling",
np_results)
pre_avg_tat, pre_avg_wt = print_table("Preemptive Priority Scheduling",
pre_results)

print("\n--- Side-by-Side Comparison ---")
print(f"{'Process':<8}{'NP-Finish':<12}{'P-Finish':<10}{'NP-WT':<8}{'P-
WT':<6}")
np_map = {r["pid"]: r for r in np_results}
pre_map = {r["pid"]: r for r in pre_results}
for pid in [p["pid"] for p in processes_base]:
    a, b = np_map[pid], pre_map[pid]
```

```
print(f"{pid:<8}{a['finish']:<12}{b['finish']:<10}{a['waiting']:<8}{b['wai
ting']:<6}")

print(f"\n{'Metric':<25}{'Non-Preemptive':<18}{'Preemptive':<12}")
print(f"{'Average Turnaround
(ms)':<25}{np_avg_tat:<18.2f}{pre_avg_tat:<12.2f}")
print(f"{'Average Waiting
(ms)':<25}{np_avg_wt:<18.2f}{pre_avg_wt:<12.2f}")

if pre_avg_wt < np_avg_wt:
    print("\n=> Preemptive scheduling gives lower average waiting time
here.")
elif pre_avg_wt > np_avg_wt:
    print("\n=> Non-Preemptive scheduling gives lower average waiting time
here.")
else:
    print("\n=> Both give the same average waiting time here.")
```

2. Output



```
--- Non-Preemptive Priority Scheduling ---
Process AT   Priority  BT   Finish  TAT   WT
P1      0    3         3    3       3     0
P2      1    2         4    7       6     2
P3      2    4         6    13      11    5
P4      3    6         4    17      14    10
P5      5    10        2    19      14    12
Average TAT = 9.60 ms | Average WT = 5.80 ms

--- Preemptive Priority Scheduling ---
Process AT   Priority  BT   Finish  TAT   WT
P1      0    3         3    7       7     4
P2      1    2         4    5       4     0
P3      2    4         6    13      11    5
P4      3    6         4    17      14    10
P5      5    10        2    19      14    12
Average TAT = 10.00 ms | Average WT = 6.20 ms

--- Side-by-Side Comparison ---
Process NP-Finish   P-Finish  NP-WT   P-WT
P1      3           7         0       4
P2      7           5         2       0
P3      13          13        5       5
P4      17          17        10      10
P5      19          19        12      12

Metric                  Non-Preemptive    Preemptive
Average Turnaround (ms)  9.60             10.00
Average Waiting (ms)     5.80             6.20

=> Non-Preemptive scheduling gives lower average waiting time he
PS C:\Users\lab\Downloads>
```

3.Discussion:

The program implements and compares **Non-Preemptive Priority Scheduling** and **Preemptive Priority Scheduling** using five processes: P1, P2, P3, P4, and P5. Each process has an Arrival Time (AT), Priority, and Burst Time (BT). In this scheduling system, a **smaller priority number indicates higher priority**.

In **Non-Preemptive Priority Scheduling**, once a process starts execution, it continues until its burst time is completed. The scheduler selects the highest-priority process from the currently available processes. From the output, the average Turnaround Time is **9.60 ms**, while the average Waiting Time is **5.80 ms**.

In **Preemptive Priority Scheduling**, a running process can be interrupted if a newly arrived process has a higher priority. The program checks the ready processes at every unit of time and executes the process with the highest priority. In this case, the average Turnaround Time is **10.00 ms** and the average Waiting Time is **6.20 ms**.

The comparison shows that **Non-Preemptive Priority Scheduling performs slightly better for this particular set of processes**. Its average waiting time is 5.80 ms compared to 6.20 ms for the preemptive method. Similarly, its average turnaround time is 9.60 ms compared to 10.00 ms.

For example, P2 has a higher priority than P1. In the preemptive method, P1 starts at time 0, but when P2 arrives at time 1 with a higher priority, P1 is interrupted. This causes P1 to finish later at time 7. On the other hand, in the non-preemptive method, P1 completes first at time 3, and then P2 executes. Therefore, the scheduling behavior and waiting times differ between the two approaches.

The results also demonstrate that **preemption does not always guarantee a lower average waiting time**. Its performance depends on the arrival time, priority, and burst time of the processes.

4. Conclusion:

In conclusion, this experiment successfully demonstrates the difference between **Non-Preemptive and Preemptive Priority Scheduling**. Both algorithms select processes based on priority, but the main difference is that preemptive scheduling allows a running process to be interrupted by a higher-priority process, while non-preemptive scheduling does not.

For the given process set, **Non-Preemptive Priority Scheduling provides better overall performance**, with an average turnaround time of **9.60 ms** and an average waiting time of **5.80 ms**. Preemptive Priority Scheduling produces an average turnaround time of **10.00 ms** and an average waiting time of **6.20 ms**.

Therefore, for this particular dataset, **Non-Preemptive Priority Scheduling is more efficient in terms of average waiting and turnaround time**. However, the best scheduling algorithm depends on the characteristics of the processes and the requirements of the operating system. In real-time or interactive systems, preemptive scheduling may still be preferable because it can respond more quickly to newly arriving high-priority processes.

5.Github Repository: https://github.com/sultan36912/Operating-System-Lab..git